Avaliação N2 - ArtGallery

Arquitetura DDD, comunicação por eventos (C4) e inspeção SOLID

Escopo: trabalho acadêmico individual baseado no código do ArtGallery. O arquivo diagramas.zip foi usado apenas como exemplo do formato de representação. Propostas não foram implementadas no aplicativo.

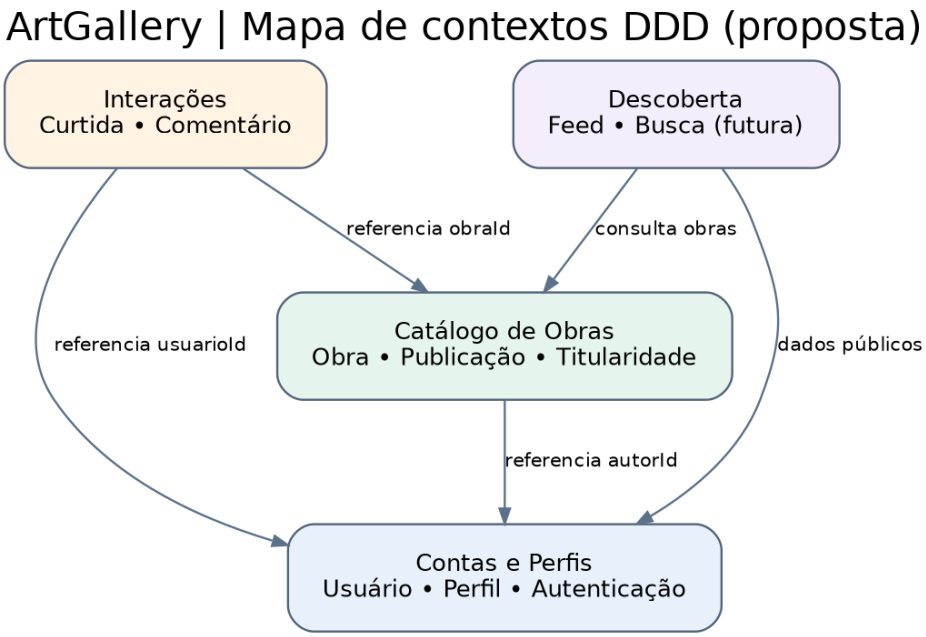
Questão 1 - Transformação para DDD (3,0)

Domínio de aplicação. O ArtGallery é uma plataforma de exposição de arte digital em que usuários criam contas e perfis, publicam obras com imagem, título e descrição, consultam uma galeria e interagem por meio de curtidas e comentários. O projeto analisado possui autenticação JWT, gerenciamento das próprias obras, contagem de visualizações e feed paginado. Utiliza Vue 3/Vite na interface, Node.js/Express na API, Sequelize/MariaDB na persistência e imagens armazenadas localmente.

Limites do que foi verificado. A busca devolve listas vazias (`searchService.js`) e as páginas administrativas são placeholders. Recursos como seguir artistas, NFT e assinatura digital não foram identificados neste código.

Contextos delimitados (Bounded Contexts)

- Contas e Perfis: Usuario, Perfil, cadastro, autenticação e edição do perfil.
- Catálogo de Obras: Obra, publicação, edição, exclusão e propriedade do autor.
- Interações: Curtida e Comentario, com vínculo por UsuarioId e ObraId.
- Descoberta: projeções de leitura do feed e futura pesquisa (ainda não funcional).

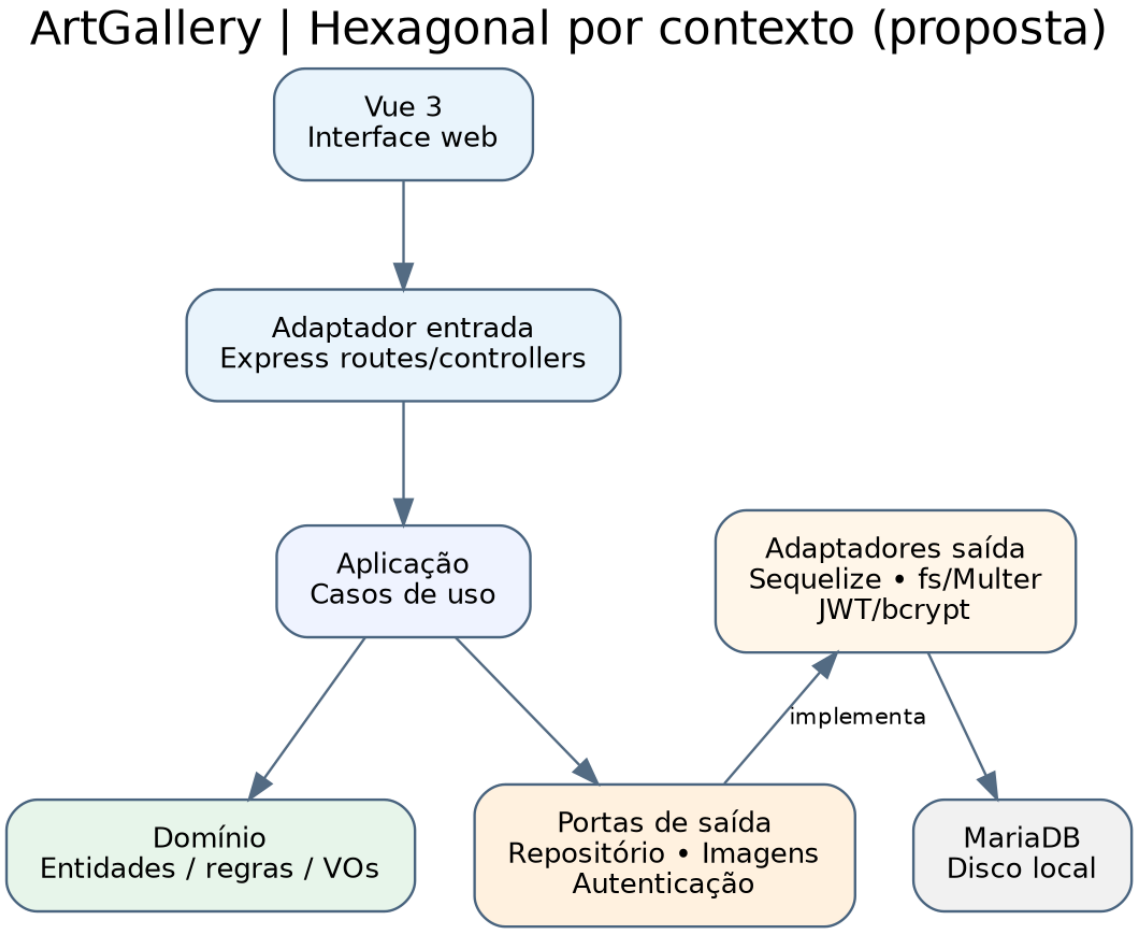


A separação é lógica, em um monólito modular; contextos não implicam microsserviços. Entidades de outros contextos são referenciadas por identificadores, não por modelos Sequelize compartilhados.

Questão 1 - Modelo tático e hexagonal

Agregados / objetos de valor propostos: Usuario (Email, NomeUsuario), Obra (TituloObra, ReferenciaImagem), Curtida (par UsuarioId/ObraId) e Comentario (ConteudoComentario). Descoberta é um modelo de consulta, não novo agregado de escrita.

Regras centrais: uma obra tem autor e imagem; somente seu autor pode alterá-la ou excluí-la; uma pessoa não pode manter duas curtidas para a mesma obra; comentários e curtidas pertencem a registros identificados; os indicadores são dados derivados.



Organização: `domain` concentra entidades e invariantes; `application`, casos de uso; `ports`, contratos de persistência, autenticação e imagens; `adapters`, Express, Sequelize, JWT/bcrypt, Multer/arquivos. Detalhes navegáveis em `arquitetura/contexts/`, no ZIP do GitHub.

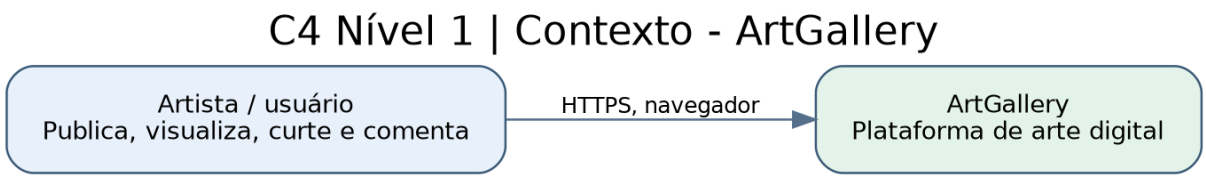
Rastreabilidade: `userService` -> Contas e Perfis; `artworkService` -> Catálogo, com leitura de Descoberta; `likeService` e `commentService` -> Interações; `searchService` -> Descoberta (apenas estrutura).

Questão 2 - Mensageria orientada a eventos (3,5)

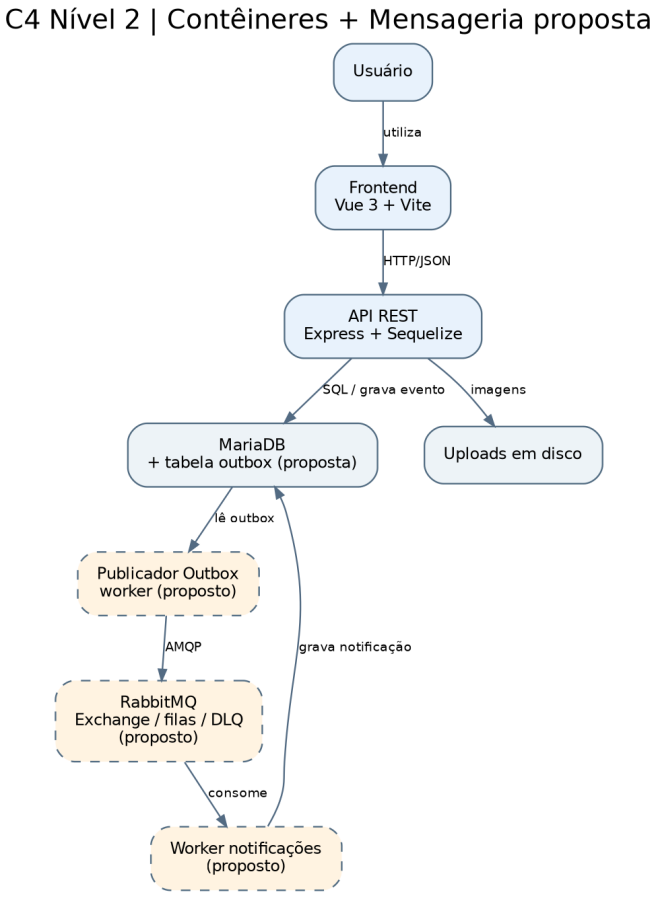
Objetivo de negócio: avisar ao artista quando outra pessoa comenta sua obra, sem atrasar a operação HTTP. A mesma infraestrutura poderá atender a eventos de obras publicadas ou curtidas.

Escolha técnica: RabbitMQ (AMQP), tabela Outbox no MariaDB, publicador assíncrono e worker de notificações. Esses componentes são propostos; não constam no código atual.

C4 Nível 1 - Contexto



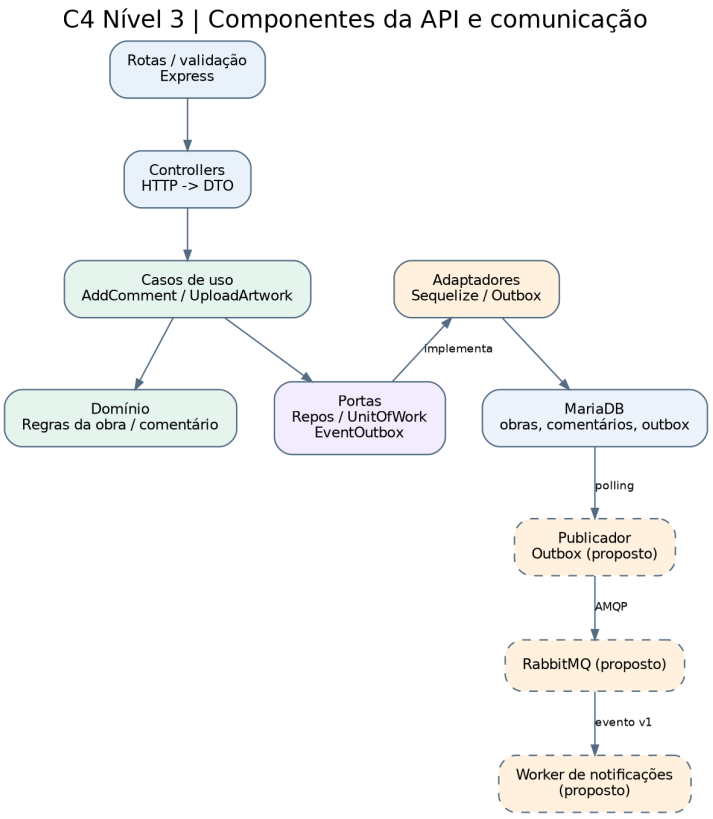
C4 Nível 2 - Contêineres



Na arquitetura atual, Vue consulta a API, que grava no MariaDB e no disco local. A proposta adiciona Outbox, publicador, broker RabbitMQ e consumidor para notificações internas.

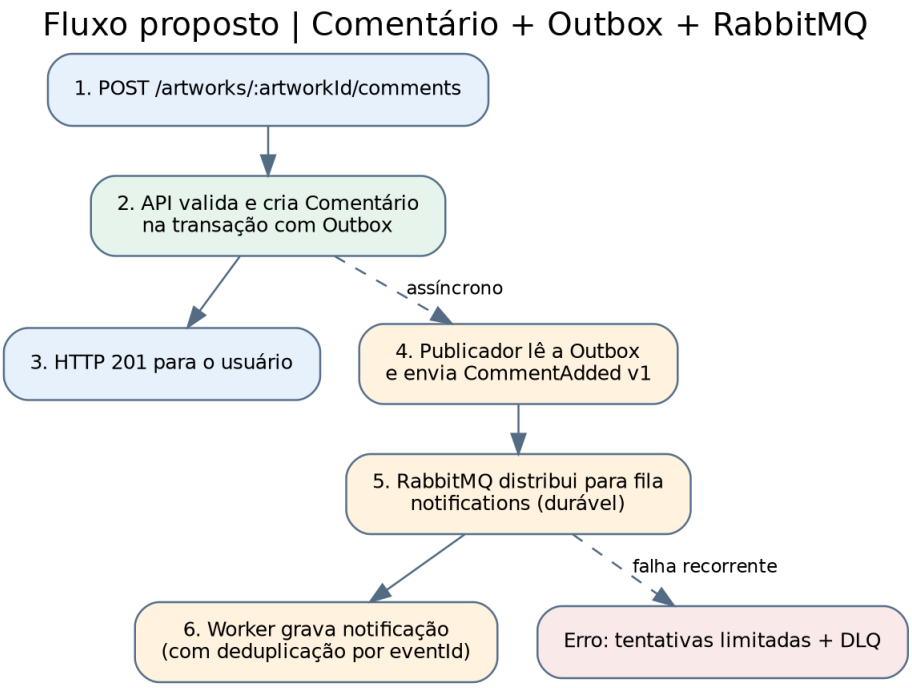
Questão 2 - Componentes e fluxo

C4 Nível 3 - API e comunicação



Evento proposto: `CommentAdded.v1`, com `eventId, occurredAt, artworkId, commentId, authorId, recipientId`. O evento não precisa carregar JWT nem dados pessoais desnecessários.

Fluxo com Outbox



Questão 2 - Justificativas

- Consistência: comentário e evento Outbox são gravados na mesma transação; se o broker falhar, o evento continua pendente.
- Desacoplamento: API responde ao usuário sem aguardar a geração da notificação.
- Confiabilidade: fila durável, confirmação manual (ack), tentativas limitadas, DLQ e consumidor idempotente por eventId.
- Evolução: novas filas podem receber outros eventos sem modificar o endpoint original.
- Trade-off: exige broker, monitoramento e aceita consistência eventual da notificação.

Implementação futura: exchange tópico `artgallery.events`, routing key `artwork.comment.added.v1` e fila `artgallery.notifications`. Quando o próprio autor comentar, não gerar notificação para si. O worker deve deduplicar o eventId antes de registrar o aviso. Não há necessidade de tornar o caminho síncrono de publicação dependente do RabbitMQ.

Questão 3 - Inspeção de SOLID (3,5)

Metodologia reprodutível: (1) extrair `ArtGallery(1).zip`; (2) navegar em `ArtGallery/art-gallery-api/modules/`, ignorando `node_modules` e `.env`; (3) ler services, controllers, imports e chamadas de infraestrutura; (4) comparar responsabilidades e dependências com SRP e DIP; (5) registrar arquivo e linhas, recortar literalmente o trecho e propor refatoração.

As seis ocorrências abaixo são indícios fundamentados de melhoria arquitetural (3 SRP, 3 DIP). Os recortes são literais, sem implementar mudanças. Não há evidência suficiente para afirmar violações de O, L ou I.

Questão 3 - Ocorrências 1 e 2

1. Atualização de perfil também remove arquivo

Arquivo: `modules/user/userService.js` (linhas 54-69) | S - Responsabilidade Única (SRP)

```
54 async function updateUserProfile(userId, fullName, bio, profilePicture) {
55   const user = await User.findByPk(userId);
56   if (!user) { const error = new Error('Usuário não encontrado.'); error.status = 404; throw error; }
57
58   const oldPicture = user.profilePicture;
59   user.fullName = fullName;
60   user.bio = bio || null;
61   if (profilePicture) user.profilePicture = profilePicture;
62   await user.save();
63
64   if (profilePicture && oldPicture && oldPicture !== 'default-profile.png' && oldPicture !== profilePicture) {
65     const oldPath = path.join(__dirname, '../../public/uploads/profiles', oldPicture);
66     if (fs.existsSync(oldPath)) fs.unlinkSync(oldPath);
67   }
68
69   return getUserProfile(userId);
```

Indício: No mesmo caso de uso são alterados dados do perfil e apagada uma imagem diretamente do disco. Persistência do usuário e ciclo de vida de arquivos são motivos diferentes para mudança.

Refatoração: Manter AtualizarPerfil como orquestrador, delegando gravação ao UsuarioRepository e limpeza de imagem à porta ImagemPerfilStorage.

2. Autenticação depende de tecnologias concretas

Arquivo: `modules/user/userService.js` (linhas 22-32) | D - Inversão de Dependência (DIP)

```
22 async function loginUser(login, password) {
23   const user = await User.findOne({ where: { [Op.or]: [{ email: login }, { username: login }] } });
24   if (!user || !(await bcrypt.compare(password, user.password))) {
25     throw new Error('E-mail ou senha inválidos.');
26   }
27
28   const safeUser = {
29     id: user.id, username: user.username, email: user.email, fullName: user.fullName,
30     profilePicture: user.profilePicture, isAdmin: user.isAdmin
31   };
32   return { token: generateToken(user), user: safeUser };
```

Indício: loginUser consulta o modelo Sequelize, compara senha via bcrypt e gera token por implementação concreta. Trocar ORM, algoritmo ou token exige mudar a regra de aplicação.

Refatoração: Injetar UsuarioRepository, SenhaHasher e TokenIssuer por contratos; colocar Sequelize, bcrypt e JWT nos adaptadores.

Questão 3 - Ocorrências 3 e 4

3. Consulta de detalhe também registra visualização

Arquivo: `modules/artwork/artworkService.js` (linhas 34-49) | S - Responsabilidade Única (SRP)

```
  34   // Uma visualização por consulta ao detalhe. currentUserId será útil depois.
  35   await artwork.increment('views');
  36   // No Sequelize v6/MySQL, increment() não garante atualizar a instância
  37   // retornada em memória. Recarregar assegura a contagem correta no JSON.
  38   await artwork.reload({
  39     include: [{
  40       model: User,
  41       attributes: ['id', 'username', 'fullName', 'profilePicture']
  42     }]
  43   });
  44   let isLiked = false;
  45   if (currentUserId) {
  46     const existingLike = await Like.findOne({ where: { userId: currentUserId, artworkId: artwork.id } });
  47     isLiked = !!existingLike;
  48   }
  49   return { artwork, isLiked };
```

Indício: A leitura de detalhe realiza escrita no contador e consulta a situação da curtida. Exibição, métrica e interação evoluem por motivos distintos.

Refatoração: Separar ConsultarDetalheObra, RegistrarVisualizacao e ConsultarCurtida; a aplicação compõe o retorno ao frontend.

4. Serviço de obras manipula diretamente o disco

Arquivo: `modules/artwork/artworkService.js` (linhas 87-100) | D - Inversão de Dependência (DIP)

```
  87 async function updateArtwork(artworkId, userId, title, description, newImageFile) {
  88   const artwork = await getArtworkForEdit(artworkId, userId);
  89   // Guardar o nome ANTES da alteração para apagar a imagem antiga só após o save.
  90   const oldImageFilename = newImageFile ? artwork.imagePath : null;
  91   artwork.title = title;
  92   artwork.description = description || '';
  93   if (newImageFile) artwork.imagePath = newImageFile.filename;
  94   await artwork.save();
  95
  96   if (oldImageFilename) {
  97     const oldImagePath = path.join(__dirname, '../../public/uploads/artworks', oldImageFilename);
  98     fs.unlink(oldImagePath, (err) => {
  99       if (err) console.error('Erro ao apagar imagem antiga:', err);
 100     });
```

Indício: updateArtwork usa path/fs para excluir imagem antiga, acoplando o caso de uso ao armazenamento local e à estrutura de pastas.

Refatoração: Introduzir porta ObraImageStorage.remove(nome) e injetar adaptador de disco; manter a política de substituição na aplicação.

Questão 3 - Ocorrências 5 e 6

5. Exclusão concentra várias responsabilidades

Arquivo: `modules/artwork/artworkService.js` (linhas 118-130) | S - Responsabilidade Única (SRP)

```
118   // Excluir o registro e atualizar o contador em uma transação no banco.
119   await sequelize.transaction(async (transaction) => {
120     await Like.destroy({ where: { artworkId: artwork.id }, transaction });
121     await Comment.destroy({ where: { artworkId: artwork.id }, transaction });
122     await artwork.destroy({ transaction });
123     await User.decrement('artworksCount', { where: { id: artwork.userId }, transaction });
124   });
125
126   // Só depois de confirmar o banco, remover o arquivo único (Grupo B).
127   const imageFilePath = path.join(__dirname, '../../public/uploads/artworks', artwork.imagePath);
128   fs.unlink(imageFilePath, (err) => {
129     if (err) console.error('Erro ao apagar imagem da obra excluída:', err);
130   });
```

Indício: deleteArtwork coordena exclusão de curtidas, comentários, obra, contador do autor e arquivo. Isso centraliza políticas de contextos distintos no serviço de obras.

Refatoração: Criar ExcluirObra (orquestração) com portas de repositório e transação; excluir arquivo com adaptador próprio e tratar compensação quando necessário.

6. Curtidas acopladas a Sequelize e modelos

Arquivo: `modules/like/likeService.js` (linhas 16-31) | D - Inversão de Dependência (DIP)

```
16 async function toggleLike(userId, artworkId) {
17   const artwork = await findArtworkOrFail(artworkId);
18   // Linha da curtida e contador da obra precisam ser gravados juntos.
19   const liked = await sequelize.transaction(async (t) => {
20     const [like, created] = await Like.findOrCreate({
21       where: { userId, artworkId: artwork.id },
22       defaults: { userId, artworkId: artwork.id },
23       transaction: t
24     });
25     if (created) {
26       await artwork.increment('likesCount', { transaction: t });
27       return true;
28     }
29     await like.destroy({ transaction: t });
30     await artwork.decrement('likesCount', { transaction: t });
31     return false;
```

Indício: toggleLike usa transação Sequelize, métodos de modelo e contador da obra diretamente. O fluxo não é testável isoladamente sem adaptações.

Refatoração: Criar AlternarCurtida com CurtidaRepository, ObraRepository e TransactionManager injetados; adaptadores Sequelize mantêm a consistência.

Conclusão: o ArtGallery tem separação inicial de rotas, controladores e serviços, mas serviços ainda acumulam responsabilidades e dependem de ORM/arquivos. A abordagem DDD/hexagonal e as refatorações propostas explicam uma possível evolução, sem afirmar que o projeto já foi migrado.

Referências: Evans (2003), Domain-Driven Design; Cockburn (2005), Hexagonal Architecture; Martin (2017), Clean Architecture; documentação C4 (c4model.com); código `ArtGallery(1).zip` enviado para análise.